

[Home](#)[GitHub](#)

CLASSES

[ErrorBoundary](#)

EVENTS

[SIGINT](#)[unhandledRejection](#)

NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

GLOBAL

[404_Error_Handler](#)[ACTIVIDADES](#)[API_URL](#)[ASPECTOS_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)[Footer](#)[GET /](#)

frontend/src/config/api.js

```
1.  /**
2.   * @file Configuración de la API y función de fetch.
3.   * @description Centraliza la URL base, los endpoints y una función
4.   */
5.
6.  /**
7.   * @constant {string} API_URL
8.   * @description La URL base del backend, obtenida de las variables de entorno.
9.   * Lee de la variable de entorno REACT_APP_API_URL.
10.  * Fallback a http://localhost:4000 solo para desarrollo local.
11.  * ⚠ IMPORTANTE: En producción, REACT_APP_API_URL DEBE estar configurado.
12.  */
13.  const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:4000';
14.
15.  /**
16.   * @namespace apiConfig
17.   * @description Un objeto que contiene la configuración de la API.
18.   * @property {string} baseUrl - La URL base del backend.
19.   * @property {object} endpoints - Un objeto con los endpoints específicos.
20.   */
21.  // Exportar la configuración
22.  export const apiConfig = {
23.    baseUrl: API_URL,
24.    endpoints: {
25.      register: `${API_URL}/api/auth/register`,
26.      login: `${API_URL}/api/auth/login`,
27.      profile: `${API_URL}/api/auth/profile`,
28.      registros: `${API_URL}/api/registros`,
29.    },
30.  };
31.
32.  /**
33.   * @function apiFetch
34.   * @description Una función de ayuda para realizar peticiones `fetch`.
35.   * @param {string} endpoint - El endpoint de la API al que se hará la petición.
36.   * @param {object} [options={}] - Opciones adicionales para la petición.
37.   * @returns {Promise<Response>} La respuesta de la petición `fetch`.
38.   * @async
39.   */
40.  // Función helper para hacer fetch con configuración por defecto
41.  export const apiFetch = async (endpoint, options = {}) => {
42.    // Obtener el estado de autenticación desde el storage de Zustand
43.    const authStorage = localStorage.getItem('auth-storage');
44.    let token = null;
45.
46.    if (authStorage) {
47.      try {
48.        const parsed = JSON.parse(authStorage);
49.        // Zustand persist guarda el estado dentro de una propiedad `state`
50.        const state = parsed.state || parsed;
51.        if (state && state.token && state.token.trim() !== '') {
52.          token = state.token;
53.          console.log('Token obtenido del localStorage:', token);
54.        } else {
```

```

55.         console.warn('⚠ Token vacío o no válido en localStor
56.     }
57.     } catch (e) {
58.         console.error("    Error parsing auth-storage from localS
59.     }
60. } else {
61.     console.warn('⚠ No se encontró auth-storage en localStorage'
62. }
63.
64. const defaultHeaders = {
65.     'Content-Type': 'application/json',
66. };
67.
68. // Si existe un token, añadirlo a los headers de autorización
69. if (token && token.trim() !== '') {
70.     defaultHeaders['Authorization'] = `Bearer ${token}`;
71.     console.log('    Authorization header añadido');
72. } else {
73.     console.warn('⚠ No Authorization header será añadido (token
74. }
75.
76. const defaultOptions = {
77.     headers: {
78.         ...defaultHeaders,
79.         ...(options.headers || {}),
80.     },
81.     mode: 'cors',
82.     ...options,
83. };
84.
85. console.log('    Realizando fetch a:', endpoint);
86. console.log('    Headers:', defaultOptions.headers);
87.
88. const response = await fetch(endpoint, defaultOptions);
89. return response;
90. };
91.
92. export default apiConfig;

```